Matriz de Riesgos — Reevaluación del Sprint 3

DrinkEmAll — Proyecto Integrador Interdisciplinar 2026
Espacio curricular: Dirección y Gestión de Proyectos Informáticos

Equipo: Senger, Brian Leonel Miguel — Baumgatner, Jose Ignacio — Riquel, Gaston Azula
Responsable del registro: Senger, Brian Leonel Miguel — Project Manager
Fecha de emisión: 23 de octubre de 2026 · **Sprint:** 3 (Semanas 7 a 9)
Reemplaza a: Matriz y Registro de Riesgos del Sprint 2 (1 de octubre de 2026)

---

1. Propósito de la reevaluación

La Tarea 3.4 del Sprint 3 encomienda *"actualizar la matriz de riesgos del proyecto analizando las dificultades surgidas en la integración de JavaScript"*.

Este documento hace tres cosas:

1. **Reevalúa los ocho riesgos** registrados en el Sprint 2, con la evidencia acumulada en las tres semanas siguientes: cuáles bajaron de prioridad porque se trataron, cuáles subieron y cuáles se mantuvieron.
2. **Analiza las dificultades concretas de la integración de JavaScript** —el consumo asincrónico del catálogo, la lógica del carrito y la validación del checkout— y deriva de ellas los riesgos nuevos.
3. **Deja la matriz consolidada** con los catorce riesgos vigentes y el orden de tratamiento para el Sprint 4.

No es un documento nuevo que reemplaza al anterior por reescritura: es la revisión prevista en el punto 5 del registro del Sprint 2, que establecía reevaluar la matriz al cierre de cada sprint.

---

2. Metodología

Se mantiene el enfoque del PMI y las escalas del Sprint 2, para que las prioridades sean comparables entre una emisión y la siguiente.

Nivel	Probabilidad	Impacto
3 — Alta	Es esperable que ocurra	Compromete la entrega o su validez
2 — Media	Puede ocurrir	Genera retrabajo significativo
1 — Baja	Improbable	Molesta, no frena

Resultado	Prioridad	Tratamiento
6 a 9	**Crítica**	Plan de acción inmediato y seguimiento semanal
3 a 4	**Media**	Plan de acción definido, revisión por sprint
1 a 2	**Baja**	Aceptado y monitoreado

Criterio de reevaluación. Un riesgo baja de prioridad únicamente cuando existe una mitigación

implementada y verificable, no cuando está planificada. Un riesgo sube cuando vuelve a materializarse o cuando un cambio del proyecto amplía su exposición.

---

3. Evolución de los riesgos del Sprint 2

ID	Riesgo	Sprint 2	Sprint 3	Movimiento	Motivo
R-01	Control de edad eludible del lado del cliente	9 Crítica	9 Crítica	=	Sin cambios en el control; la exposición creció
R-02	Divergencia de ramas e integración tardía	9 Crítica	9 Crítica	=	El atraso pasó de 16 a 25 *commits*
R-03	La interfaz no resiste el volumen real del catálogo	6 Crítica	4 Media	▼	Filtro de marcas con buscador y paginación en servidor
R-04	Concentración del trabajo cerca de la entrega	6 Crítica	9 Crítica	▲	Se repitió y se agravó en el Sprint 3
R-05	Configuración de entorno que impide levantar el proyecto	6 Crítica	3 Media	▼	Entorno reproducible en contenedores
R-06	Dependencia del backend para validar el frontend	6 Crítica	2 Baja	▼	API desplegada permanentemente y respaldo local reforzado
R-07	Ausencia de pruebas automatizadas	4 Media	6 Crítica	▲	Hay más lógica con reglas de negocio sin cubrir
R-08	Material de terceros con copyright en el repositorio	3 Media	3 Media	=	Control vigente y verificado en cada *commit*

Tres riesgos bajaron, dos subieron y tres se mantuvieron. Los que bajaron lo hicieron por trabajo efectivamente hecho; los que subieron, por hechos verificables ocurridos durante el sprint.

---

4. Fichas de los riesgos reevaluados

R-01 · Control de edad eludible del lado del cliente

Probabilidad: Alta (3) · **Impacto:** Alto (3) · **Prioridad: 9 (Crítica)** · *Sin variación*

La verificación de mayoría de edad sigue resolviéndose en el navegador y guardándose en el almacenamiento local. Durante el Sprint 3 el checkout pasó a registrar la conformidad junto con los datos del cliente y la forma de entrega, lo que mejora la trazabilidad del registro, pero **no cambia la naturaleza del control**: se elude borrando el almacenamiento del navegador.

Qué cambió. La tienda está publicada en internet desde el 5 de octubre. En el Sprint 2 el riesgo era

teórico porque el sitio corría en las máquinas del equipo; hoy cualquier persona con el enlace llega al circuito de compra. La probabilidad no sube porque ya era alta, pero la exposición es otra.

Plan de acción. Validar la conformidad del lado del servidor cuando el checkout se conecte a la API, rechazando allí todo pedido sin constancia. Estimado en 6 horas y asentado como trabajo diferido en el Registro de Solicitudes de Cambio.

Contención vigente. La venta es simulada: no hay pasarela de pago ni confirmación real de pedido. Mientras eso sea así, el riesgo legal efectivo es acotado, y la limitación está declarada por escrito en la Política de Privacidad y en el propio código.

R-02 · Divergencia de ramas e integración tardía

Probabilidad: Alta (3) · **Impacto:** Alto (3) · **Prioridad: 9 (Crítica)** · *Sin variación, con empeoramiento*

La rama `main` acumula **25 *commits*** de atraso** respecto de `dev`, frente a los 16 registrados al cierre del Sprint 2. El trabajo de los Sprints 2 y 3 completos —documentación, despliegue, carrito, detalle de producto, consumo asincrónico y validación del checkout— vive únicamente en `dev`.

Por qué no bajó. No se hizo ninguna integración en el período. La decisión de no integrar fue deliberada y está registrada, pero la deuda sigue creciendo y cada día que pasa vuelve más costoso el merge.

Plan de acción. Integrar `dev` a `main` al inicio del Sprint 4, antes de agregar funcionalidad nueva, y no al cierre. Estimado en 3 horas.

Plan de contingencia. Si la integración presenta conflictos extensos, resolverlos en una rama de integración dedicada y verificar que el proyecto instale, compile y levante desde cero antes de fusionar.

R-03 · La interfaz no resiste el volumen real del catálogo

Probabilidad: Media (2) · **Impacto:** Medio (2) · **Prioridad: 4 (Media)** · *Baja desde 6*

El riesgo original era concreto: el filtro de marcas renderizaba una casilla por marca, y con casi cuatrocientas marcas reales el panel lateral era inusable.

Mitigación implementada y verificable. El panel lista las ocho marcas con más productos y agrega un buscador cuando hay más, conservando siempre visibles las marcas ya elegidas. El filtrado, el ordenamiento y la paginación se resuelven en el servidor, de modo que la interfaz nunca carga el catálogo completo. El sitio en producción sirve hoy 1.183 productos y 393 marcas —una menos que en la emisión anterior— sin degradarse.

Riesgo residual. Las imágenes del catálogo pesan 59,5 MB y se sirven en un único tamaño, sin versiones adaptativas. En conexiones lentas la experiencia se resiente. Está registrado como trabajo diferido al Sprint 4.

R-04 · Concentración del trabajo cerca de la fecha de entrega

Probabilidad: Alta (3) · **Impacto:** Alto (3) · **Prioridad: 9 (Crítica)** · *Sube desde 6*

Riesgo materializado por segunda vez. El historial del repositorio muestra el mismo patrón que

motivó el registro original, agravado: un *commit* el 5 de octubre, **dieciocho días sin actividad** y todo el trabajo técnico del Sprint 3 —consumo asincrónico, validación del checkout y los tres documentos de gestión— concentrado el 23 de octubre, el mismo día de la entrega.

Por qué sube el impacto. En el Sprint 2 la concentración produjo apuro; en el Sprint 3 dejó al proyecto sin margen de corrección: cualquier problema aparecido en la jornada final —un error de compilación, una dependencia que no instala, el servidor caído— habría significado entregar incompleto, sin día siguiente disponible.

Plan de acción. Pasar de hitos por sprint a **hitos por semana con evidencia verificable**: cada viernes debe existir al menos un *commit* que cierre una tarjeta del Kanban. El plan de casos de prueba de este sprint aporta el criterio objetivo para declarar terminada una tarjeta.

Plan de contingencia. Ante atraso detectado el primer viernes sin evidencia, recortar alcance de forma explícita y declararlo —como se hizo con las tres solicitudes de cambio rechazadas o diferidas— en lugar de confiar en recuperar el tiempo sobre la fecha límite.

R-05 · Configuración de entorno que impide levantar el proyecto

Probabilidad: Baja (1) · **Impacto:** Alto (3) · **Prioridad: 3 (Media)** · *Baja desde 6*

Mitigación implementada. El despliegue en contenedores dejó una instalación de referencia que funciona y no depende de la máquina de nadie: imágenes con la versión de Node y del gestor de paquetes fijadas, dependencias instaladas desde el archivo de bloqueo y base de datos con sus migraciones aplicadas al arrancar. Ante una máquina local que no levanta, hay contra qué comparar.

Por qué el impacto sigue alto. Si ocurre, bloquea al integrante afectado por completo, como ya pasó con el gestor de paquetes inexistente declarado en una rama.

R-06 · Dependencia del backend para validar el frontend

Probabilidad: Baja (1) · **Impacto:** Medio (2) · **Prioridad: 2 (Baja)** · *Baja desde 6*

Mitigación implementada y reforzada. La API corre de forma permanente en el servidor desplegado, de modo que ya no hace falta que nadie la levante. Además, durante este sprint los cuatro *hooks* del catálogo se reescribieron para que, cuando la API no responde, **caigan al archivo estático en lugar de quedar vacíos**; antes ese respaldo solo existía cuando la API no estaba configurada.

Estado. Riesgo aceptado y monitoreado. Se mantiene en el registro porque el servidor es único y una caída suya volvería a exponerlo.

R-07 · Ausencia de pruebas automatizadas

Probabilidad: Media (2) · **Impacto:** Alto (3) · **Prioridad: 6 (Crítica)** · *Sube desde 4*

Sigue sin existir una sola prueba automatizada. Lo que cambió es la cantidad de lógica expuesta: al carrito y a la adaptación de datos se sumaron, en este sprint, siete reglas de validación del checkout y cuatro *hooks* asincrónicos con estados de carga, error y cancelación.

Por qué sube el impacto. Son reglas que fallan en silencio. Una validación que deja de rechazar un

correo mal formado, o un *hook* que muestra el producto anterior mientras carga el nuevo, no rompen la compilación ni la pantalla: pasan inadvertidos hasta la demostración.

Mitigación parcial de este sprint. La lógica de validación se escribió **separada del componente**, en un módulo de reglas puras, precisamente para que pueda probarse sin renderizar nada; se verificó con 30 casos ejecutados a mano. El catálogo asincrónico se verificó con un guion que simula la red y comprueba que los *hooks* compartan una sola petición y que un fallo permita reintentar. Ninguna de las dos verificaciones quedó automatizada.

Plan de acción. El Plan de Casos de Prueba emitido en este sprint deja los casos especificados; automatizarlos en el Sprint 4 es transcribirlos, no diseñarlos. Estimado en 18 horas, diferido con fundamento.

R-08 · Material de terceros con copyright dentro del repositorio

Probabilidad: Baja (1) · **Impacto:** Alto (3) · **Prioridad: 3 (Media)** · *Sin variación*

La copia local de 198 MB de la plantilla comercial de referencia sigue en el directorio de trabajo, excluida por `.gitignore` y verificada en cada *commit*. No se filtró ningún archivo en los 26 *commits* del repositorio.

Qué cambió. El repositorio es público y el proyecto está licenciado bajo GPL-3.0. Una filtración ahora no solo redistribuiría obra ajena: lo haría bajo una licencia incompatible con ella.

---

5. Dificultades surgidas en la integración de JavaScript

Esta sección responde al pedido expreso de la consigna. Las dificultades que siguen son las que efectivamente aparecieron al programar el consumo asincrónico, el carrito y la validación, y son el origen de los riesgos nuevos de la sección 6.

5.1. La lectura del catálogo dejó de ser sincrónica. Hasta este sprint el archivo `productos.json` se incorporaba al código en tiempo de compilación, de modo que los datos estaban disponibles desde el primer dibujado. Al pasar a la Fetch API, los cuatro *hooks* del catálogo tuvieron que reescribirse con estado de carga, de error y de cancelación. Lo que era una constante pasó a ser una operación que puede tardar, fallar o quedar obsoleta.

5.2. Varios *hooks* pedían el mismo archivo a la vez. La cabecera, el panel de filtros, la grilla y el *hero* necesitan el catálogo simultáneamente: la implementación directa producía cuatro descargas del mismo archivo. Se resolvió memorizando la promesa, de modo que las cuatro llamadas compartan un único pedido de red, y borrándola si falla para que el siguiente intento vuelva a pedirlo en lugar de repetir el error para siempre.

5.3. Respuestas que llegan desordenadas. Con peticiones concurrentes, una respuesta lenta puede llegar después de otra más nueva y pisarla: el caso típico es escribir en el buscador y ver los resultados de una búsqueda anterior. Se trató con `AbortController` para cancelar la petición en curso y con una bandera de vigencia que descarta la respuesta de un efecto ya desmontado.

5.4. No se puede marcar "cargando" de cualquier manera. Las reglas de *hooks* del proyecto impiden modificar el estado de forma sincrónica dentro de un efecto, porque provoca un dibujado en cascada. En la ficha de producto eso obligó a una solución distinta: el identificador pedido viaja **dentro** del estado, y mientras no coincida con el de la dirección se considera que está cargando. Sin ese detalle, al cambiar de producto se mostraba el anterior durante un instante.

5.5. La validación no podía vivir dentro del componente. Las reglas del checkout se escribieron en un módulo aparte, de funciones puras, para poder ejecutarlas sin dibujar nada y para que el formulario se ocupe solo de mostrar los errores, no de decidirlos. Es la condición que hace posible automatizar esas pruebas en el Sprint 4.

5.6. El despliegue encadenó cinco problemas no previstos. Límite de descargas del registro público de imágenes, gestor de paquetes distinto al relevado, dos dependencias con *scripts* de compilación bloqueados por política del gestor y un cliente de base de datos que exigía cifrado y rechazaba el certificado autofirmado. Ninguno era previsible desde el código; todos consumieron tiempo de la jornada de despliegue.

---

6. Riesgos nuevos

R-09 · Condiciones de carrera en el consumo asincrónico

Categoría: Técnico · **Probabilidad:** Baja (1) · **Impacto:** Medio (2) · **Prioridad: 2 (Baja)**

Con peticiones concurrentes, una respuesta vieja puede sobrescribir a una más nueva y mostrar datos que no corresponden a lo que la persona pidió.

Plan de acción. Ya implementado: cancelación con `AbortController`, bandera de vigencia en cada efecto y comparación del identificador pedido contra el del estado. La probabilidad es baja porque el tratamiento está en el código, no en el plan.

Plan de contingencia. Ante un caso detectado, reproducirlo frenando la red desde las herramientas del navegador antes de intentar corregirlo a ciegas.

R-10 · El catálogo pasa a depender de una descarga de red

Categoría: Técnico · **Probabilidad:** Baja (1) · **Impacto:** Alto (3) · **Prioridad: 3 (Media)**

Al salir del código compilado, `productos.json` dejó de estar garantizado: ahora es un archivo que se pide por red y puede faltar por un despliegue incompleto, una ruta mal resuelta si el sitio se publica en un subdirectorio, o una caché servida con contenido viejo. Sin API configurada y sin ese archivo, la tienda no tiene productos que mostrar.

Plan de acción. La ruta se construye respetando la base de publicación del sitio, y el fallo de descarga borra la promesa memorizada para permitir reintentos. Verificado: el archivo se sirve con código 200 y tipo `application/json`, y tras un error simulado el siguiente intento se recupera.

Plan de contingencia. Incluir la descarga del archivo en la lista de verificación posterior a cada despliegue, junto con la carga de la portada.

R-11 · La validación del checkout solo existe del lado del cliente

Categoría: Técnico / Legal · **Probabilidad:** Alta (3) · **Impacto:** Medio (2) · **Prioridad: 6 (Crítica)**

Las siete reglas de validación —nombre, apellido, correo, teléfono, dirección, ciudad y código postal— se

ejecutan en el navegador. Cualquiera puede saltearlas con las herramientas de desarrollo y enviar un pedido con datos inconsistentes. Es el mismo problema de fondo que R-01, aplicado a los datos de entrega.

Plan de acción. Repetir la validación en el servidor cuando el checkout se conecte a la API, usando las mismas reglas como especificación. El módulo de reglas está escrito de forma independiente del navegador, de modo que puede reutilizarse.

Plan de contingencia. Mientras la validación de servidor no exista, mantener el pedido como simulación y no habilitar despacho ni cobro reales.

R-12 · El carrito vive en el almacenamiento del navegador

Categoría: Técnico · **Probabilidad:** Media (2) · **Impacto:** Medio (2) · **Prioridad: 4 (Media)**

El contenido del carrito se guarda localmente, sin servidor. Eso permite recuperar la compra tras recargar la página, pero trae tres consecuencias: se pierde al cambiar de dispositivo o de navegador, puede quedar vacío si el navegador bloquea el almacenamiento, y **puede conservar precios o existencias desactualizados** si el catálogo cambió desde que el producto se agregó.

Plan de acción. Revalidar precio y existencia contra la fuente de datos al entrar al checkout, antes de mostrar el total. Toda lectura y escritura del almacenamiento ya está protegida para que una excepción no deje la tienda sin arrancar.

Plan de contingencia. Si se detecta una diferencia de precio, informarla explícitamente en el checkout en lugar de corregirla en silencio.

R-13 · Dependencia de infraestructura externa y recursos compartidos

Categoría: Técnico / Infraestructura · **Probabilidad:** Alta (3) · **Impacto:** Medio (2) · **Prioridad: 6 (Crítica)**

Riesgo materializado. El despliegue dependió de servicios de terceros que fallaron o impusieron límites: el registro público de imágenes cortó las descargas por cuota agotada, y el cliente de base de datos de la distribución elegida exigía cifrado. Además, el servidor comparte memoria con otros servicios del propietario, por lo que los contenedores corren con límites de memoria ajustados.

Plan de acción. El prefijo del registro de imágenes quedó parametrizado, de modo que cambiar de origen es una variable de entorno y no una edición del archivo de construcción. Los límites de memoria están declarados en la configuración de despliegue y se revisan si se agrega un servicio.

Plan de contingencia. Ante una caída del servidor durante una demostración, mostrar la tienda desde el entorno local, que funciona con el catálogo estático sin depender de la API.

R-14 · El tablero no refleja el estado real del trabajo

Categoría: Gestión · **Probabilidad:** Media (2) · **Impacto:** Medio (2) · **Prioridad: 4 (Media)**

Riesgo materializado. Durante la auditoría de cierre de este sprint se detectó que el *issue* #6 —pruebas automatizadas, límite de error y estados de carga— figuraba **cerrado sin que el trabajo se hubiera hecho**. Un tablero que declara terminado lo que no existe invalida cualquier medición de avance y oculta

deuda.

Plan de acción. Regla incorporada al circuito de control de cambios: un *issue* solo se cierra con el *commit* que lo resuelve referenciado. Revisión del tablero contra el historial del repositorio al cierre de cada sprint.

Plan de contingencia. Ante una discrepancia, reabrir la tarjeta y asentarla como deuda en el registro de cambios, como se hizo con el *issue* #6.

---

7. Riesgos materializados durante el Sprint 3

ID	Qué ocurrió	Resolución
R-04	Dieciocho días sin actividad y todo el trabajo técnico concentrado el día de la entrega	Sin resolver: se eleva a prioridad 9 y se fijan hitos semanales
R-13	Cinco problemas encadenados de infraestructura durante el despliegue	Resueltos: registro parametrizado, dependencias habilitadas explícitamente y conexión sin cifrado al contenedor de base de datos
R-14	*Issue* #6 cerrado sin trabajo realizado	Detectado en la auditoría de cierre; se reabre y queda asentado como deuda

Los riesgos R-02, R-05 y R-06, materializados en el Sprint 2, no volvieron a ocurrir en este período.

---

8. Matriz consolidada

ID	Riesgo	Categoría	Prob.	Imp.	Prior.
R-01	Control de edad eludible del lado del cliente	Legal / Técnico	3	3	**9 Crítica**
R-02	Divergencia de ramas e integración tardía	Equipo	3	3	**9 Crítica**
R-04	Concentración del trabajo cerca de la entrega	Tiempos	3	3	**9 Crítica**
R-07	Ausencia de pruebas automatizadas	Técnico	2	3	**6 Crítica**
R-11	Validación del checkout solo del lado del cliente	Técnico / Legal	3	2	**6 Crítica**
R-13	Dependencia de infraestructura externa y recursos compartidos	Infraestructura	3	2	**6 Crítica**
R-03	La interfaz no resiste el volumen real del catálogo	Técnico	2	2	**4 Media**
R-12	El carrito vive en el almacenamiento del navegador	Técnico	2	2	**4 Media**
R-14	El tablero no refleja el estado real del trabajo	Gestión	2	2	**4 Media**
R-05	Configuración de entorno que impide levantar el proyecto	Técnico	1	3	**3 Media**

ID	Riesgo	Categoría	Prob.	Imp.	Prior.
R-08	Material de terceros con copyright en el repositorio	Legal	1	3	**3 Media**
R-10	El catálogo depende de una descarga de red	Técnico	1	3	**3 Media**
R-06	Dependencia del backend para validar el frontend	Equipo / Técnico	1	2	**2 Baja**
R-09	Condiciones de carrera en el consumo asincrónico	Técnico	1	2	**2 Baja**

Distribución: 6 riesgos críticos, 6 medios y 2 bajos, sobre un total de 14.

---

9. Mapa de calor

Impacto \ Probabilidad	1 — Baja	2 — Media	3 — Alta
3 — Alto	R-05, R-08, R-10	R-07	R-01, R-02, R-04
2 — Medio	R-06, R-09	R-03, R-12, R-14	R-11, R-13
1 — Bajo	—	—	—

El cuadrante superior derecho concentra los tres riesgos de prioridad 9. Dos de ellos —R-02 y R-04— no son técnicos sino de disciplina de trabajo, y son los que más condicionan el Sprint 4.

---

10. Orden de tratamiento para el Sprint 4

Prioridad	Riesgo	Acción comprometida	Estimación
1	R-02	Integrar `dev` a `main` antes de agregar funcionalidad nueva	3 h
2	R-04	Hitos semanales con evidencia verificable en el repositorio	—
3	R-01 y R-11	Validar edad y datos del checkout del lado del servidor	6 h
4	R-07	Automatizar los casos del Plan de Casos de Prueba	18 h
5	R-03	Optimizar imágenes del catálogo con tamaños adaptativos	12 h
6	R-12	Revalidar precio y existencia al entrar al checkout	4 h

Las estimaciones son del Project Manager y coinciden con las registradas como trabajo diferido en el Registro de Solicitudes de Cambio del Sprint 3.

---

11. Seguimiento

La matriz se revisa al cierre de cada sprint. En cada revisión corresponde:

1. Reevaluar probabilidad e impacto de los riesgos abiertos, bajando de prioridad únicamente los que tengan mitigación implementada y verificable.

2. Registrar los que se materializaron y cómo se resolvieron.
3. Incorporar los nuevos que aparezcan.
4. Dar de baja los que dejaron de aplicar, dejando constancia del motivo.

Próxima revisión: cierre del Sprint 4.

Documento emitido por: Senger, Brian Leonel Miguel — Project Manager **Fecha:** 23 de octubre de 2026 **Revisión:** 2.0 · Reevaluación del Sprint 3